

# Sistema de gestión del V Congreso Argentino de Agroecología

---

Trabajo Final Integrador — **Grupo 1**

De la maqueta al sistema desplegado: seis entregas, una aplicación en producción.

**Integrantes:** Benjamín Rodríguez Mantilla · Alcides Castro · Lucas Budnik

Jakarta EE 11

Jersey 3.1

Hibernate 7 · MySQL

Angular 19

Docker + GitLab CI

# El equipo y cómo nos repartimos hoy

1-2 min

## Benjamín Rodríguez Mantilla

Modelo de datos, persistencia y servicios REST ·  
Inscripciones, pagos y usuarios

**Hoy presenta:** apertura, entregas 2 a 4 y balance técnico

## Alcides Castro

Flujo académico y frontend Angular · Evaluación, agenda y  
programa

**Hoy presenta:** entregas 1, 5 y 6, y la demo en vivo

## Lucas Budnik

Autenticación y roles, certificados y despliegue · CI/CD y  
pruebas

**Hoy presenta:** el problema, la arquitectura, la organización  
y el cierre

6

ENTREGAS

208

COMMITTS

~54k

LÍNEAS DE CÓDIGO

3

MESES DE TRABAJO

# 01

## El problema

Qué congreso hay que gestionar, cómo nos acercamos y para quién construimos.

# Un congreso no es un formulario: son cinco procesos en paralelo

2 min

El **V Congreso Argentino de Agroecología** (FCAyF–UNLP) reúne durante varios días a cientos de personas: docentes, productores, estudiantes y organizaciones. Hoy eso se coordina con planillas, mails y formularios sueltos.

## Inscribirse y pagar

Categorías con aranceles distintos, transferencia o efectivo, comprobantes, recibos y facturas.

## Enviar y evaluar trabajos

Resúmenes por eje temático, revisión por pares a ciegas, dictámenes, desempates y devoluciones.

## Armar el programa

Mesas, pósters, talleres y conferencias en aulas con cupo y franjas horarias que no pueden solaparse.

## Comunicar

Circulares, avisos por etapa y recordatorios: hoy, listas de correo copiadas a mano.

## Certificar

Asistencia, presentación y evaluación. Al cierre, cada persona debe poder descargar lo suyo.

## El costo de no tenerlo

Datos duplicados, plazos que se pasan, trabajos sin evaluador y certificados armados uno por uno.

# Cómo nos acercamos al problema

- **Leímos el congreso real, no el enunciado.** Revisamos las circulares y las ediciones anteriores del Congreso Argentino de Agroecología para entender ejes temáticos, modalidades y plazos.
- **Modelamos el proceso como etapas con fechas.** Descubrimos que casi toda regla del sistema es en realidad “¿en qué etapa estamos?": envío, evaluación, inscripción, certificación.
- **Escribimos historias de usuario por rol** antes de escribir una línea de Java. Cada historia dijo qué pantalla hacía falta.
- **Bocetamos y validamos las pantallas** con un prototipo navegable, sin backend. Nos permitió discutir el flujo con el grupo antes de comprometer el modelo.

## La decisión que ordenó todo

No hay “estados mágicos”: el congreso tiene **etapas configurables** ( `EtapaProceso` + `RangoFechas` ) y el administrador las define. El sistema habilita o bloquea acciones según la fecha, en lugar de tener reglas escritas a mano en el código.

Lo mismo con los **catálogos**: ejes, modalidades y tipos de envío los configura el comité, no el programador.

# Usuarios y adoptantes de la solución

6 roles

| ROL                                          | QUÉ HACE EN EL SISTEMA                                                                  | POR QUÉ LE IMPORTA                                                |
|----------------------------------------------|-----------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| Asistente                                    | Se registra, se inscribe, paga, arma su cronograma personal y descarga certificados.    | Deja de mandar mails para saber si su pago fue aprobado.          |
| Autor                                        | Carga trabajos con coautores, sube PDF/DOCX, envía y recibe la devolución.              | Ve el estado real de su trabajo y el dictamen sin intermediarios. |
| Evaluador                                    | Acepta asignaciones dentro de su cupo por eje y emite dictamen.                         | Recibe sólo trabajos de su especialidad y nunca el propio.        |
| Comité académico<br>(organizador científico) | Precheck, asignación de evaluadores, desempates, catálogos y plazos.                    | Es el <b>adoptante principal</b> : hoy hace todo esto a mano.     |
| Administración                               | Usuarios, aranceles, pagos y arqueo, aulas y programa, circulares, cierre del congreso. | Concentra la operación y la trazabilidad del dinero.              |
| Público                                      | Consulta programa, circulares e historia sin iniciar sesión.                            | Es la cara visible del congreso.                                  |

# 02

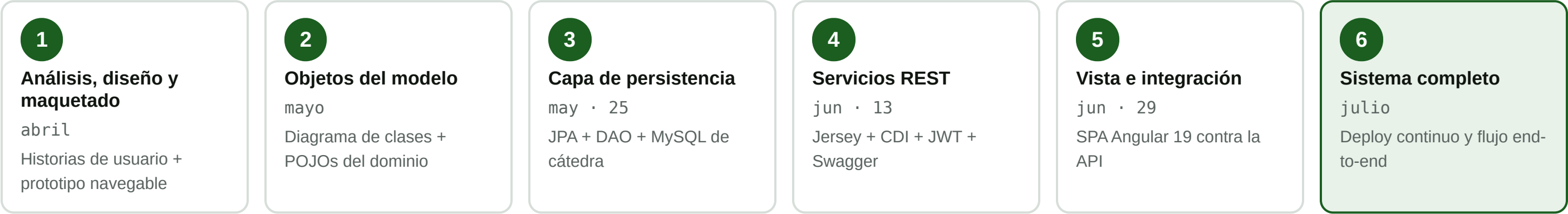
## Las seis entregas

Qué objetivo tuvo cada etapa, qué conceptos de la cátedra aplicamos, con qué nos peleamos y qué quedó funcionando.



# El camino: de la maqueta al sistema desplegado

abril → julio 2026



## Una regla que nos salvó

Cada entrega vivió en su propia rama (`entrega-3`, `entrega-4`, `entrega-5`) y se integró a `main` sólo con el pipeline en verde. Nunca perdimos una entrega ya aprobada por trabajar sobre la siguiente.

208 commits

9 ramas

278 clases Java

82 componentes Angular

23 entidades JPA

26 recursos REST



# Análisis, diseño y maquetado

Producto: historias de usuario + prototipo navegable

## Objetivo

Entender el dominio y acordar *qué* íbamos a construir, con pantallas discutibles en lugar de descripciones verbales.

## Qué aplicamos

Relevamiento por rol, historias de usuario, arquitectura cliente–servidor y separación en capas (teoría de Inyección de Dependencias, lámina de capas).

## Herramientas

Figma Make para el bocetado; un prototipo **React + Vite** con datos en `localStorage`, sin backend.

## Dificultad

Al maquetar aparecieron reglas que no estaban en el enunciado: ¿qué pasa si dos evaluadores empatan?, ¿puede un evaluador revisar su propio trabajo? **Cómo la resolvimos:** las anotamos como reglas de negocio explícitas y las llevamos al modelo de la Entrega 2 en lugar de improvisarlas después.



### CAPTURA 1 — Bocetado / historias de usuario

Una pantalla del prototipo React junto al tablero de historias de usuario por rol. Guardar como `img/e1-maqueta.png`.

El prototipo de la Entrega 1 sigue en el repositorio: fue la referencia visual de todo lo que vino después.

# Definición de los objetos del modelo

Producto: diagrama de clases + dominio en Java

### Objetivo

Traducir las historias a un modelo de objetos: entidades, enums y relaciones, sin pensar todavía en la base de datos.

### Qué aplicamos

Diseño OO, POJOs, enumerados para estados, y el principio de **responsabilidad única**: la lógica del congreso no vive en las entidades sino en `SistemaGestionCongresoService`.

### Herramientas

UML + Maven para estructurar el proyecto Java (`pom.xml`, empaquetado `war`, Java 21).

### Dificultad

Modelar la evaluación por pares. Una asignación no es una simple relación entre trabajo y evaluador: tiene aceptación, rechazo, dictamen y desempate. **Cómo la resolvimos**: la convertimos en entidad propia, `AsignacionEvaluacion`, con su `Evaluacion` asociada. Esa decisión sostuvo todas las entregas siguientes.



### CAPTURA 2 — Fragmento del diagrama de clases

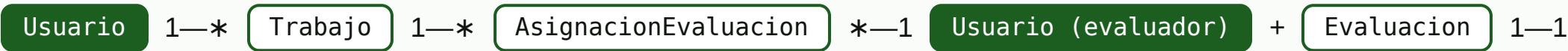
Recortar el núcleo `Usuario · Trabajo · AsignacionEvaluacion · Evaluacion` del `Diagrama de Clases Grupo01.pdf`. Guardar como `img/e2-diagrama.png`.

Elegimos el fragmento más denso del diagrama: el que muestra la evaluación por pares.

# Las relaciones que hacen complejo al dominio

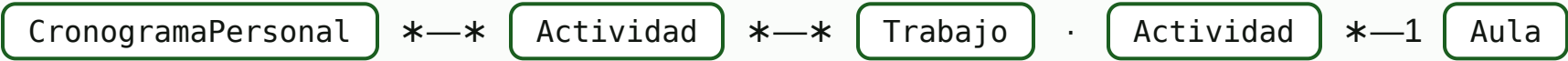
23 entidades

## 1 · Evaluación por pares



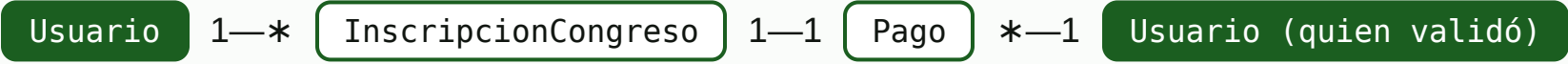
Con `EvaluadorEjeCapacidad` limitando cuántos trabajos por eje puede tomar cada evaluador.

## 2 · Agenda del congreso



Dos `@ManyToMany` encadenadas: por eso el chequeo de solapamiento y de cupo del aula fue la validación más difícil.

## 3 · Inscripción y dinero



El pago guarda **quién** lo validó: es lo que hace auditable el arqueo de caja.

## 4 · Configuración del congreso

`Congreso` usa `@ElementCollection` de embeddables anidados: `EtapaCongreso` → `RangoFechas`, más los catálogos de ejes, modalidades y tipos de envío (`CatalogoItem`).

# Capa de persistencia

Producto: 23 entidades JPA + DAOs + ABM verificable

## Objetivo

Que el modelo sobreviva al reinicio del servidor, sobre la base MySQL de la cátedra ( jyaa2026\_bd1 ).

## Qué aplicamos (JPA · DAO · Listeners)

@Entity , @OneToMany / @ManyToMany , @Embeddable , @ElementCollection , EntityManager , JPQL con paginado ( setFirstResult / setMaxResults ) y el patrón DAO: GenericDAO<T> → AbstractJpaDAO<T> → un DAO por entidad.

## Librerías y herramientas

Hibernate 7.0, mysql-connector-j 9.2, Maven, Eclipse + Tomcat 10, phpMyAdmin de la cátedra. El arranque lo hace un ServletContextListener ( JpaBootstrapListener ) que crea el EntityManagerFactory y carga datos iniciales.

## Dificultades reales

- ① El WAR desplegaba pero no encontraba las entidades: hubo que listarlas explícitamente en persistence.xml , porque Tomcat no hace el escaneo automático de un servidor Jakarta EE completo.
- ② Los PDF de trabajos no entraban en la columna: pasamos el @Lob a LONGLOB .
- ③ hbm2ddl.auto=update nunca achica ni renombra columnas, así que escribimos una clase SchemaMigration con SQL propio que corre al arrancar.



## CAPTURA 3 — ABM funcionando

El servlet /test-persistencia con los casos de alta/baja/modificación en verde, o las tablas creadas en phpMyAdmin. Guardar como img/e3-persistencia.png .

Servlet de pruebas: la forma más rápida de demostrar que el ABM sobre JPA realmente escribe en MySQL.

# Capa de servicios REST

Producto: API documentada en OpenAPI + JWT

### Objetivo

Exponer el dominio como una API HTTP consumible por cualquier cliente, con autenticación y errores de negocio legibles.

### Qué aplicamos (REST · CDI · Filtros · CORS)

**JAX-RS con Jersey:** `@Path`, verbos HTTP, códigos de estado, `@QueryParam` / `@PathParam`, multipart para archivos y **DTOs** separados de las entidades.

**CDI con Weld** sobre Tomcat: `@Inject`, `@RequestScoped`, y un `@Produces` / `@Disposes` de `EntityManager` que abre y commitea la transacción por request.

**Filtros JAX-RS:** `JwtAuthFilter`, `CorsRequestFilter` (`@PreMatching`, preflight `OPTIONS`) y `CorsResponseFilter`.

**ExceptionMappers** que convierten `NegocioException` en un JSON con mensaje para el usuario.

### Librerías

Jersey 3.1.9 (+ `jersey-cdi1x-servlet`, `media-json-jackson`, `media-multipart`), Weld 6.0.4, JWT 0.12.6, `swagger-jaxrs2-jakarta` 2.2.28, Angus Mail.

### Dificultad

Hacer convivir **Jersey + CDI en Tomcat**: sin el puente `jersey-cdi1x-servlet` y un `beans.xml` con `bean-discovery-mode="annotated"`, las inyecciones en los recursos llegaban en `null`. **Cómo la resolvimos:** siguiendo la teoría de Inyección de Dependencias, movimos el `EntityManager` a un *producer* con scope de request.



### CAPTURA 4 — Swagger UI

La lista de recursos desplegada y un `POST /api/login` ejecutado con su respuesta. Guardar como `img/e4-swagger.png`.

26 recursos REST documentados con `@Tag` y `@Operation`, publicados como OpenAPI.

# Sesión con JWT: por qué no usamos HttpSession

## Cómo funciona

- `POST /api/login` devuelve un **JWT firmado con HS256**, válido 4 horas.
- El cliente lo manda en `Authorization: Bearer ...` en cada pedido.
- `JwtAuthFilter` lo valida y arma un `AuthenticatedUser` para el recurso.
- Rutas públicas declaradas: login, registro, `health`, programa, circulares y OpenAPI.
- En el cliente: `sessionStorage` + cierre por inactividad a los 30 minutos.

## El detalle que aprendimos peleándonos

Un token es una foto del momento en que se emitió. Si el administrador te quita un rol o te inhabilita la cuenta, el token viejo **seguiría siendo válido** durante horas.

**Nuestra solución:** el filtro valida la firma del token, pero vuelve a leer **roles y estado activo desde la base** en cada request. Así, inhabilitar una cuenta la expulsa al instante, sin esperar el vencimiento.

**Por qué JWT y no `HttpSession`:** la API tiene que poder servir a un cliente Angular alojado en otro origen y a un backend replicado en contenedores. Sin estado en el servidor, el escalado y el CORS se simplifican.

# Capa de presentación e integración

Producto: SPA Angular 19 contra la API real

### Objetivo

Reemplazar el prototipo con datos falsos por un cliente real que consuma el REST y respete los permisos de cada rol.

### Qué aplicamos (Angular 1, 2 y 3)

**Componentes standalone** y `bootstrapApplication`; ruteo con `app.routes.ts`; **guards** de sesión y de rol; un **interceptor funcional** que agrega el `Bearer` y desloguea ante un 401; `HttpClient` con genéricos tipados; **formularios reactivos** y la nueva sintaxis de control de flujo `@if` / `@for`.

### Librerías y herramientas

Angular 19 + Angular CLI, RxJS, Angular Material (diálogos), **Leaflet** para el mapa de sede y aulas, `jwt-decode`. En desarrollo, `proxy.conf.json` apunta `/api` al servidor de la cátedra: se programa sin Tomcat local.

### Dificultades reales

- ① **CORS** en desarrollo: el navegador bloqueaba las llamadas. Dos caminos, y usamos los dos — proxy de Angular para el día a día, filtros CORS en el backend para probar contra el servidor desplegado.
- ② **NG0203**: llamar a `inject()` dentro de `ngOnInit` rompe la app; hubo que moverlo al campo de la clase.
- ③ El **bundle superaba el presupuesto** de producción y el build fallaba en el pipeline: ajustamos los budgets y depuramos importaciones.
- ④ Leaflet se recreaba en cada ciclo de detección de cambios y el mapa parpadeaba.



### CAPTURA 5 — La SPA y la red

Un panel de la app con el DevTools abierto en Network mostrando la llamada `/api/...` con su header `Authorization`. Guardar como `img/e5-angular.png`.

La captura muestra las dos capas a la vez: la pantalla y el REST que la alimenta.



# Sistema completo

Producto: aplicación en producción, ciclo de vida completo del congreso

### Objetivo

Cerrar el ciclo completo — de la inscripción al certificado — y dejarlo desplegado y usable por los organizadores.

### Qué sumamos en esta etapa

Aranceles configurables por categoría con QR · recibo de caja correlativo y arqueo diario · factura al participante · cupos de evaluador por eje con liberación automática · desempate 1–1 que dispara un tercer evaluador · catálogos editables por el comité · programa con aulas georreferenciadas y control de cupo · circulares y notificaciones por plantilla (in-app + email) · checklist pre-congreso · cierre del congreso que emite los certificados.

### Integración final

Un **Dockerfile multi-stage** compila Angular, lo copia dentro de `src/main/webapp`, empaqueta el WAR con Maven y lo despliega en Tomcat 10 / JDK 21. Frontend y API terminan en el **mismo origen**: en producción ya no hace falta CORS.

### Dificultad

El contenedor a veces se desplegaba con un frontend viejo por la caché de Docker. **Cómo la resolvimos:** build con `--no-cache --pull` y, sobre todo, **pasos de verificación dentro del propio Dockerfile**: si el WAR no contiene el `main-*.js` y los estilos esperados, la imagen no se construye. El error se detecta en el pipeline, no en la demo.



### CAPTURA 6 — Sistema en producción

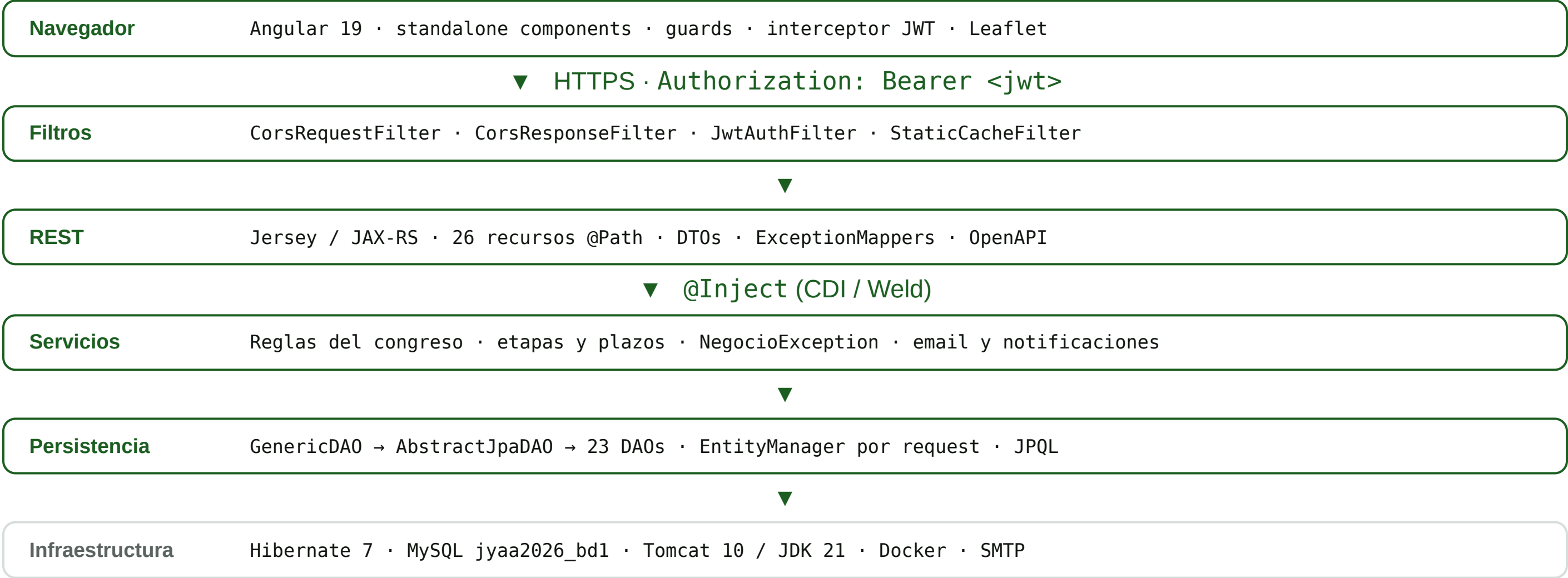
Home pública en `grupo1.jyaa-ci.linti.unlp.edu.ar` o el programa del congreso con el mapa de aulas. Guardar como `img/e6-produccion.png`.

Desplegado y accesible: cada push a `main` lo reconstruye.



# La arquitectura que resultó de las seis entregas

Un solo WAR



# Cada tema de la cátedra, en un archivo concreto del proyecto

14 clases teóricas

| TEORÍA                              | DÓNDE VIVE EN EL PROYECTO                         | TEORÍA                               | DÓNDE VIVE EN EL PROYECTO                             |
|-------------------------------------|---------------------------------------------------|--------------------------------------|-------------------------------------------------------|
| Arquitectura server-side y Servlets | PersistenciaTestServlet · web.xml · WAR en Tomcat | Inyección de dependencias (CDI/Weld) | EntityManagerProducer · @Inject · beans.xml           |
| Sesiones                            | JWT 4 h + idle 30 min (en vez de HttpSession)     | REST                                 | JaxRsApplication · @Path · verbos · ExceptionMappers  |
| ServletContext y Listeners          | JpaBootstrapListener → EMF + datos iniciales      | Swagger / OpenAPI                    | OpenApiResource · @Tag · @Operation                   |
| Filtros                             | JwtAuthFilter · StaticCacheFilter · filtros CORS  | CORS                                 | CorsRequestFilter @PreMatching (preflight OPTIONS)    |
| Maven                               | pom.xml · packaging war · build multi-stage       | Angular — parte 1                    | standalone components · app.routes.ts · servicios     |
| JDBC, Datasource y patrón DAO       | GenericDAO → AbstractJpaDAO → 23 DAOs             | Angular — parte 2                    | HttpClient tipado · formularios reactivos · RxJS      |
| JPA                                 | 23 @Entity · @ElementCollection · JPQL paginado   | Angular — parte 3                    | guards de rol · interceptor JWT · build de producción |

No quedó ningún tema de la cursada sin usar: el trabajo final los integró a todos en un mismo despliegue.

# 03

## Cómo trabajamos

La organización interna del grupo y las herramientas que la sostuvieron.

# Nos repartimos por flujo del congreso, no por capa

3 integrantes

Al principio dividimos por capas —uno el backend, otro el front— y chocábamos todo el tiempo en los mismos archivos. A partir de la Entrega 3 cambiamos: **cada uno se hizo dueño de un flujo completo**, de la entidad JPA a la pantalla Angular.

## Benjamín

Inscripciones, pagos, aranceles, arqueo y facturación · gestión de usuarios · el grueso del modelo JPA y los recursos REST.

157 commits

## Alcides

Flujo académico: envío de trabajos, comité, asignaciones, dictámenes y desempates · programa, aulas, cronogramas y mapas.

42 commits

## Lucas

Autenticación y roles · certificados automáticos · circulares y notificaciones · despliegue, pipeline y pruebas de integración.

8 commits

## Lo que funcionó

Ser dueño de un flujo de punta a punta bajó los conflictos de merge casi a cero y hizo que cada uno tuviera que aprender **todas** las capas.

## Lo que nos costó

Los cambios en `Usuario` y en la configuración del congreso tocan a los tres. Los coordinamos antes de codear y los integramos primero.

# Git, GitLab, Docker y despliegue continuo

push a main → app actualizada

- **Git con dos remotos:** GitHub para el trabajo diario del grupo y **GitLab de la cátedra** como remoto de entrega y despliegue.
- **Una rama por entrega** ( entrega-3 ... entrega-5 ) más ramas de feature por persona; `main` siempre desplegable.
- **Docker multi-stage:** etapa Node que compila Angular, etapa Maven que arma el WAR, imagen final Tomcat 10 / JDK 21.
- **GitLab CI** con runner `jyaa` : construye, detiene y elimina el contenedor anterior y levanta el nuevo detrás de Traefik con TLS automático.
- **Verificación en el build:** el Dockerfile comprueba el contenido del WAR antes de publicar la imagen.

Git

GitLab CI

Docker

Traefik + TLS

Maven

Eclipse / VS Code

phpMyAdmin

Swagger UI

Postman / DevTools

## El pipeline, en cuatro pasos

- **build** — `docker buildx build --no-cache :`  
Angular + WAR frescos y verificados
- **test** — se detiene el contenedor `grupo1` anterior
- **test** — se elimina el contenedor viejo
- **deploy** — `docker run` con las etiquetas de Traefik para `grupo1.jyaa-ci.linti.unlp.edu.ar`



## CAPTURA 7 — Pipeline en verde

Los cuatro jobs de GitLab CI completados. Guardar como `img/ci-pipeline.png`.

# 04

## Demo y reflexión

El sistema funcionando de punta a punta, y qué nos llevamos del proceso.

# Un trabajo, de principio a fin

6 min · grupo1.jyaa-ci.linti.unlp.edu.ar

- **1 · Público** — home, programa del congreso con el mapa de aulas y circulares, sin iniciar sesión.
- **2 · Asistente** — registro, inscripción por pasos, pago por transferencia con comprobante.
- **3 · Administración** — aprueba el pago, se emite el recibo correlativo y llega la notificación.
- **4 · Autor** — carga un trabajo con coautores, sube el PDF y lo envía.
- **5 · Comité** — precheck, asigna dos evaluadores respetando el cupo del eje.
- **6 · Evaluador** — acepta la asignación y emite el dictamen.
- **7 · Autor** — ve la devolución en su panel.
- **8 · Administración** — cierra el congreso y se habilitan los certificados.

## Lo que queremos que se vea

No las pantallas sueltas, sino que **una acción de un rol cambia lo que ve otro rol**, en la misma base de datos y en tiempo real.

## Plan B

Si la red falla: video de respaldo del recorrido completo en `img/demo-respaldo.mp4` y capturas de cada paso.

## Preparado de antemano

Sesiones abiertas en pestañas separadas por rol, congreso con etapas vigentes y un trabajo listo para enviar.



# Las cinco dificultades que más nos enseñaron

y cómo las resolvimos

| DIFICULTAD             | QUÉ PASABA                                                                                                           | CÓMO LA RESOLVIMOS                                                                                                                             |
|------------------------|----------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| CDI en Tomcat          | Las inyecciones en los recursos Jersey llegaban en <code>null</code> : Tomcat no trae CDI.                           | Weld + <code>jersey-cdi1x-servlet</code> + <code>beans.xml</code> , y el <code>EntityManager</code> como <i>producer</i> con scope de request. |
| CORS                   | El navegador bloqueaba las llamadas del Angular local al backend desplegado.                                         | Proxy de Angular para desarrollo y filtros CORS con preflight para el resto. En producción, mismo origen: el problema desaparece.              |
| Esquema de base        | <code>hbm2ddl.auto=update</code> agrega columnas pero nunca las corrige; los PDF no entraban y un campo quedó corto. | Clase <code>SchemaMigration</code> con SQL propio que corre al arrancar la aplicación, más <code>LONGLOB</code> para los documentos.           |
| Deploy con front viejo | La caché de Docker publicaba un WAR con un Angular desactualizado.                                                   | <code>--no-cache --pull</code> y verificaciones del contenido del WAR dentro del Dockerfile: el pipeline falla antes de desplegar.             |
| Trabajo en paralelo    | Conflictos constantes al dividirnos por capas, y <code>node_modules</code> versionado por error.                     | Reparto por flujo funcional, <code>.gitignore</code> saneado y una rama por entrega que sólo se integra con el pipeline en verde.              |



# Qué nos llevamos

2 min

## Las capas no son burocracia

Cuando cambiamos el frontend entero de React a Angular, el backend no se tocó. Ese día entendimos para qué sirve separar en capas.

## El modelo de datos es la decisión más cara

Convertir la asignación de evaluadores en una entidad propia costó una tarde en la Entrega 2 y nos ahorró semanas en la 4 y la 6.

## Configurable le gana a hardcodeado

Etapas, catálogos, aranceles y cupos los define el organizador. Si estuvieran en el código, cada edición del congreso pediría un despliegue.

## Desplegar temprano y seguido

Tener CI desde la Entrega 3 convirtió “anda en mi máquina” en un problema de cinco minutos en vez de una crisis la noche anterior a entregar.

## Programar en equipo es coordinar, no repartir

Git resuelve los archivos; las decisiones sobre el dominio compartido hay que conversarlas antes de escribirlas.

## Si tuviéramos una entrega más

Hash de contraseñas con BCrypt, autorización declarativa con `@RolesAllowed`, Bean Validation en los DTOs y pruebas automatizadas dentro del pipeline.

# Gracias. ¿Preguntas?

---

Sistema de gestión del V Congreso Argentino de Agroecología

**Aplicación:** grupo1.jyaa-ci.linti.unlp.edu.ar    **API:** /api/health · **OpenAPI:** /api/openapi.json

**Repositorio:** GitLab de la cátedra — grupo 1